DHIRUBHAI AMBANI UNIVERSITY

IT305 Computer Networks Lab

Fault Tolerant Topic-based File Distribution Service

Socket Programming Project: Final Report

Project formed by Group 097 and Group 112

`FinalReport_Group_097_112`

Group members

Name	Roll number	Contribution (confirm actual work)
Aryan Dalsaniya	202401405	Part I single-threaded server, topic discovery and integration.
Harshil Shah	202401468	Client protocol, file reception and integrity checks.
Dhvanit Shah	202401467	Part I multithreaded server and concurrency experiments.
Dwij Patel	202401418	Part II Case 1: fault injection and restart-from-beginning recovery.
Tanishq Shah	202401474	Part II Case 2: session management, checkpoints and automated tests.
Niranjan Panchal	202401441	Part II Case 2 Enhanced: parallel connections and performance analysis.

Names and roll numbers follow the submitted design document. Contribution descriptions are a proposed allocation and must be confirmed by the group.

1. Overview and test setup

This report describes the topic-based file distribution service and the experiments run on it. A client asks a server for a topic with GET, and the server sends back every regular file in the directory that belongs to that topic. Part I does this with a single-threaded and a multithreaded server. Part II keeps the multithreaded server and makes the transfer survive dropped connections: Case 1 starts over, Case 2 resumes from a checkpoint, and Case 2 Enhanced adds parallel TCP connections.

Table 1. Source folders

Folder	What it contains
`Part1/` (single mode)	Part I server that serves one client at a time, and the client.
`Part1/` (multi mode)	Part I server with one thread per connection, timing counters and the same client.
`Part2_Case1/`	Fault injection with no retained transfer checkpoint; the client restarts the topic.
`Part2_Case2/`	Checkpoint recovery with client state, a server session table and validated resumption.
`Part2_Case2_Enhanced/`	Configurable parallel TCP connections, each able to resume independently.

Everything is written in C++17 using POSIX sockets and POSIX threads, and built with `g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -pthread`. Running `make` in each of the four folders produces `server` and `client` in that folder. The required command lines are:

`./server <port> <topics_root_dir>` (Part I)

`./server <port> <topics_root_dir> <failure_probability>` (Part II)

`./client <server_ip> <server_port> <topic> <output_dir>`

No path, port or IP address is hard-coded. File data moves in chunks of at most 64 KiB. An optional `single` argument selects sequential Part I service; the default is `multi`. The enhanced client defaults to four connections and accepts an optional connection count.

1.1 Test setup

Every performance number in this report comes from actual runs on one Linux host, with server and clients as separate processes communicating over `127.0.0.1`. The environment recorded Linux 6.18.55, GCC 13.3.0, Python 3.12.14 and nine visible logical CPUs. These results measure local TCP behavior; the required three-computer LAN experiment has not been performed.

The measured topic is the complete `cane` directory from the original Animals10 dataset: 4,863 images, 60,686,716 bytes (57.88 MiB). The archive was downloaded from `https://www.kaggle.com/datasets/alessiocorrado99/animals10`. All 87 measured runs across 29 configurations have three repetitions and SHA-256 verification. Archive provenance and every source-file hash are included in `Results/Animals10/`. Earlier synthetic tests are retained separately in `Results/Local/`.

2. Part I: basic topic-based file distribution

2.1 Topics and directories

The `topics_root_dir` command-line argument contains one subfolder per topic. At startup, the server builds an in-memory map from each folder name to its canonical directory path and a sorted list of regular files. A file record includes the name, size, inode, device and modification time. Topic names are case-sensitive; source symbolic links and subdirectories are skipped.

The server rejects empty names, dot components, path separators and NUL bytes. It checks source metadata before opening a transfer file. The dataset must remain unchanged during a server run; new topics or changed files require restarting the server. This follows the startup-mapping approach in the group's submitted design.

2.2 Message format

All cases use the same framed binary protocol. Each frame starts with a 20-byte header: magic (4 bytes), type (4), session/transaction ID (8) and payload length (4). Integers are big-endian. A string has a four-byte byte count followed by its bytes, so filenames can contain spaces. A frame payload is limited to 16 MiB.

Table 2. Part I messages

Direction	Message	Meaning
Client to server	GET	Topic, worker assignment and initial file/offset cursor.
Server to client	OK	Ordered manifest containing every filename and size.
Client to server	START	Client accepts the manifest and requests the data.
Server to client	FILE_HDR	File index, starting offset, full size and name.
Server to client	DATA	File index, offset, CRC32 and up to 64 KiB of file bytes.
Client to server	ACK	Confirms the cursor after a write or file transition.
Server to client	FILE_END	Current file is complete; its transition is acknowledged.
Server to client	TRANSFER_END	All files assigned to this worker are complete.
Server to client	ERROR	Invalid topic, inconsistent request or another failure.

The receiver uses the frame length to know how many bytes to read. The `sendall` and `recvall` loops handle short operations and interrupted system calls; one TCP read is never assumed to equal one application message. The file payload is binary and is not interpreted as text.

2.3 Single-threaded server

The single-threaded server has a plain accept loop. It accepts one connection, serves the complete topic, closes the connection and returns to accept. Other connections may wait in the listen backlog, but their file transfers are served sequentially. This implementation supplies the performance baseline.

2.4 Multithreaded server and client

The multithreaded server uses the same protocol. Its main thread accepts connections and creates a POSIX worker thread for each. Each worker owns its socket, transfer buffer and counters. The read-only topic map is shared; locks protect session-table changes and log output. SIGPIPE is ignored, so a disconnected client does not terminate the server. At most 256 workers are active at once.

The client connects, requests the topic, creates `output_dir/topic/` and receives each file. It validates the index, offset, length and CRC32 before writing. Unsafe filenames, destination symbolic links and hard-linked destination files are refused. Each client in an experiment has a separate output directory. Elapsed time uses a monotonic clock; CSV output records payload, application bytes, attempts and useful throughput.

2.5 Performance measurement and results

`Tools/benchmark_animals10.py` launches N clients concurrently and records time from the first launch until the last process exits. The common batch time includes setup, retries and process overhead, but excludes server startup and SHA-256 checking. Aggregate useful throughput is the sum of useful bytes divided by that time. Tables show means across three runs; $\pm$ denotes sample standard deviation where included.

Table 3. Part I aggregate throughput on cane (60,686,716 bytes per client, loopback)

Clients	Total payload (bytes)	Control bytes	Time, multi (s)	Multi (MiB/s)	Single (MiB/s)
1	60,686,716	1,352,050	1.9466	29.75	30.13
2	121,373,432	2,704,100	2.1363	54.19	28.07
4	242,746,864	5,408,200	2.3717	97.61	27.91
8	485,493,728	10,816,400	3.5736	129.58	29.27

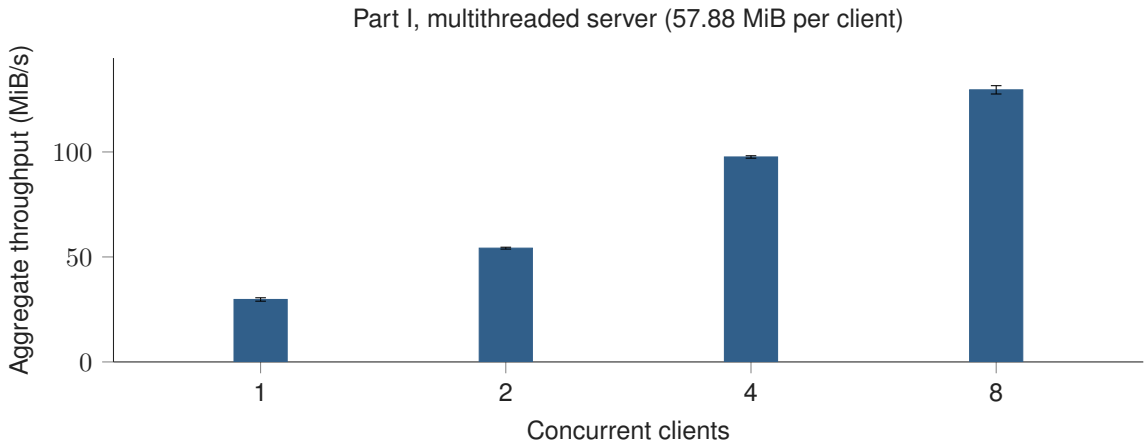


Figure 1. Aggregate throughput of the multithreaded server; error bars show sample standard deviation over three runs.

2.6 Analysis

The multithreaded server averaged 29.75 MiB/s with one client and 129.58 MiB/s with eight. Single-threaded means were 30.13 and 29.27 MiB/s, respectively. At eight clients, the ratio of multithreaded to single-threaded mean throughput was 4.43. All clients requested the same complete image topic.

The host shares CPU, filesystem and memory resources between the server and clients. Small image files incur a manifest, file header, data ACK and file-transition exchange; this workload differs from a single large-file transfer. Variation across the three repetitions is shown by the error bars. These observations do not establish the throughput of a physical network link.

Without failed payload, payload send rate equals useful throughput. Application control overhead was 1,352,050 bytes for one client and 10,816,400 bytes for eight. The counters include both directions and cover framing, manifests and ACKs. Throughput and overhead graphs are included in `Results/Animals10/figures/`.

3. Part II: fault-tolerant file transfer

Part II builds on the multithreaded server, with one thread per active connection in every case. The server takes the third required argument, `failure_probability`, and deliberately drops connections. The cases differ in how they recover after a drop.

3.1 Fault injection

The failure decision is made after each DATA chunk has been sent and its ACK has been received. The serving thread draws a value in $[0, 1)$ from `std::mt19937_64`; if it is below p, the server closes that connection. Each connection has its own stream, derived from a startup seed and its connection number. An optional seed supports repeatable stream generation, although concurrent connection ordering can vary.

The same decision point and 64 KiB chunk limit apply in all Part II cases. The draw can happen after the last data chunk of a file, before FILE_END. A probability of zero disables injection. Values outside $[0, 1]$ are rejected. Empty files and empty topics have no DATA decision point. Controlled proxy tests separately exercise failures during headers, within DATA, and at terminal messages.

The topic has 4,863 DATA chunks. One restart attempt succeeds with probability $(1 - p)^M$. At $p = 0.0003084$, this is approximately 0.2231, implying about 4.48 attempts under independent trials. The six rates were declared before measurement: 0.0000000, 0.00003084, 0.0000771, 0.0001542, 0.0002313, 0.0003084. They are identical across cases and scaled for the full topic. At $p = 1$, a nonempty restart transfer cannot finish.

For result analysis, let U be useful bytes, T total file payload sent over every attempt, A all application bytes sent in both directions, and t batch time. Redundant payload is $T - U$, application control overhead is $A - T$, and useful throughput is $U/(t\,2^{20})$ MiB/s. Sent bytes are bytes accepted by socket send calls. These counters do not include Ethernet/IP/TCP headers or kernel TCP retransmissions.

3.2 Case 1: no session management

3.2.1 Design

Case 1 keeps no transfer checkpoint between attempts. The server is the Part I multithreaded service with fault injection and the same GET, OK, START, FILE_HDR, DATA and completion exchanges. Every new connection requests a fresh transfer.

The client wraps the request in a retry loop, with 1,000 attempts by default. On reconnect it resets its cursor to the first file, validates the manifest and removes previously downloaded files belonging to that manifest before sending START. Thus an interruption in a late file causes earlier completed files to be transferred again. Retry delay starts at 5 ms and increases by 5 ms per attempt up to 100 ms. Measurement counters include work from failed attempts.

3.2.2 Results

Table 4. Case 1 on cane at $p = 0.0003084$ (means of three completed runs)

Measure	Value
Attempts	4.33
Injected failures	3.33
Useful payload	60,686,716 bytes
Total payload sent	132,898,224 bytes
Redundant payload	72,211,508 bytes
Application control overhead	3,453,659.33 bytes
Redundant share of sent payload	54.34%
Bytes sent per useful byte	2.19
Completion time	4.0822 s
Useful throughput	17.49 MiB/s

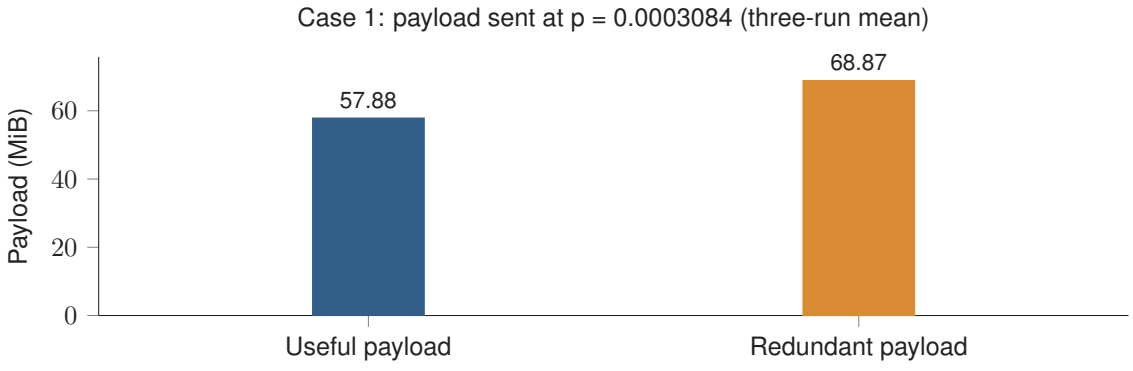


Figure 2. Useful and redundant payload sent by the Case 1 server.

3.2.3 Analysis

At $p = 0.0003084$, Case 1 sent a mean 132,898,224.33 payload bytes to deliver 60,686,716 useful bytes. Redundant payload was 72,211,508.33 bytes, or 118.99% of useful payload and 54.34% of total sent payload. The transfer sent about 2.19 bytes per useful byte. These percentages use different denominators.

Mean completion time was 4.0822 s with injection at this rate and 1.9812 s with injection disabled. At $p = 0.0002313$, mean injected failures were 2.33, mean completion time was 3.6270 s and redundant payload was 91.70% of useful payload. Times include repeat work, setup and retry delays.

The raw logs retain every measured configuration, including low-failure runs. A finite random sample need not produce a strictly monotonic timing curve. The largest probability was selected from topic length before execution; it was not chosen after inspecting throughput. Fault counts, repeated bytes and elapsed time must be considered together when interpreting restart cost.

3.3 Case 2: session management with checkpoint and resume

3.3.1 Design

Table 5. Case 2 checkpoint-related messages

Direction	Message	Meaning
Client to server	GET with session ID	Starts a session or resumes the existing topic/worker assignment at a file-index and byte-offset cursor.
Server to client	OK, then FILE_HDR	Returns the manifest and starts the expected file at the validated offset.
Server to client	DATA	Carries the next chunk and its CRC32. The server records the offered successor before sending.
Client to server	ACK	Confirms the client's cursor after recording a complete chunk or completing a file.
Server to client	FILE_END, TRANSFER_END	Completes a file or the worker's transfer. FILE_END has its own ACK.
Server to client	ERROR	Rejects an unknown topic, mismatched assignment or invalid checkpoint; BUSY is retried.

Client checkpoint. The checkpoint is an in-memory record containing the session ID, topic, worker assignment, file index and byte offset. The client advances it only after a complete DATA frame passes CRC32 and its bytes are written successfully with `pwrite`. It then sends ACK. Unlike a durable checkpoint file, this record survives a dropped connection but not a restart of the client process.

Server validation. The server maintains a mutex-protected table of session records. Each record stores the topic and worker assignment, a committed cursor and the last offered successor. On reconnect, the client reports its session ID and cursor. Only the committed cursor or that exact offered successor is accepted. An arbitrary offset, wrong assignment or expired noninitial session is rejected.

This distinction handles a lost ACK. If the client wrote a chunk but its ACK disappeared, its cursor is the offered successor and the server can commit it during reconnection. If the connection broke in the middle of a frame, the client still reports the committed cursor and the incomplete chunk is sent again. Only uncommitted data may need retransmission.

File boundaries. FILE_END is acknowledged with the next assigned file index, including for empty files. A loss of FILE_END, its ACK or TRANSFER_END is recoverable. On reopening an interrupted file, the client truncates any uncommitted tail. Server sessions remain in memory, with 600-second idle retention and a 10,000-record limit; neither endpoint provides process-crash recovery.

3.3.2 Results

Table 6. Case 2 on cane (60,686,716 useful bytes, three runs per probability)

$p\,(10^{-4})$	Injected failures	Redundant bytes	Control bytes	Time (s)	MiB/s
0.000	0.00	0	1,352,050	1.9355	29.93
2.313	1.33	0	1,657,047	2.0373	28.41
3.084	1.33	0	1,657,047	2.0480	28.27

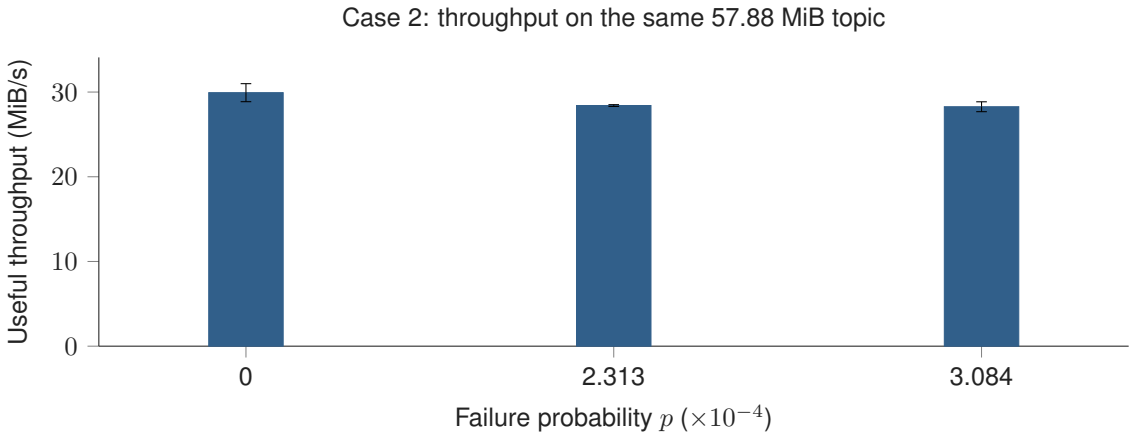


Figure 3. Case 2 throughput; error bars show sample standard deviation. Mean failure counts are listed in Table 6.

Every completed benchmark download matched all 4,863 source SHA-256 values. Under the built-in post-ACK fault policy, Case 2 sent exactly 60,686,716 payload bytes at every measured probability. Separate integration checks cover lost DATA ACKs and recovery of incomplete frames; all 28 checks passed. The integration logs are in `Results/Validation/`.

Table 7. Case 1 and Case 2 on the same topic at the same $p = 0.0003084$ (three-run means)

Measure	Case 1	Case 2
Attempts	4.33	2.33
Payload sent (bytes)	132898224	60686716
Redundant bytes	72211508	0
Application control bytes	3453659.33	1657047.33
Completion time (s)	4.0822	2.0480
Useful throughput (MiB/s)	17.49	28.27

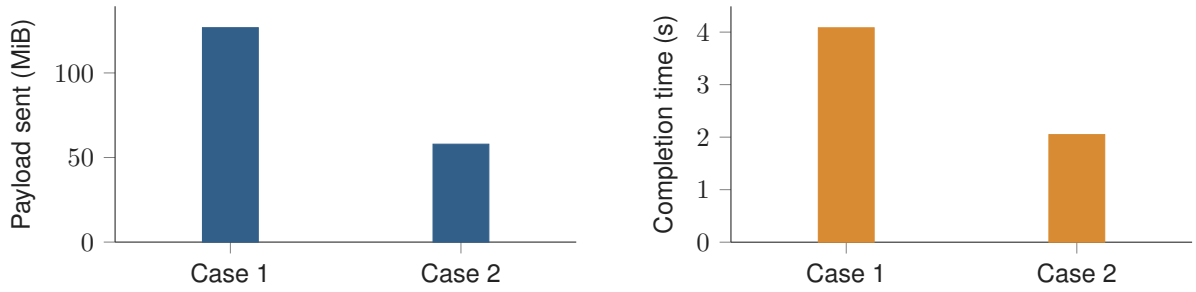


Figure 4. Payload sent and completion time for restart and resume on cane, at $p = 0.0003084$.

3.3.3 Analysis

At $p = 0.0003084$, Case 2 completed in a mean 2.0480 s versus 4.0822 s for Case 1; the ratio of mean times was 1.99. Mean useful throughput was 28.27 and 17.49 MiB/s, respectively. Checkpoints avoided repeating recorded file data under this injection policy; reconnects, manifests, ACKs and retry pauses still incurred costs.

The no-redundancy result depends on injection occurring after ACK. A break during a frame may require an uncommitted chunk to be repeated. Clean Case 2 averaged 29.93 MiB/s, compared with 29.75 MiB/s for Part I multi mode with one client. Both use the same Animals10 topic, but three shared-host repetitions do not isolate a precise checkpoint-management cost.

3.4 Case 2 Enhanced: parallel connections

3.4.1 Design

The main enhancement is parallel TCP connections: one logical client opens K connections for a topic and runs a worker thread on each. The default is four, with an optional client argument allowing one to 32. Blocking sockets are retained on both endpoints. A worker blocked in `send` or `recv` holds up its own connection, while the other workers can continue.

Table 8. Case 2 Enhanced messages

Direction	Message	Meaning
Client to server	GET(topic, worker, K, cursor)	Opens or resumes this worker's session. The frame carries its independent session ID.
Server to client	OK(manifest)	Sends the complete ordered manifest. Assignment is determined from file index and worker count.
Client to server	START	Requests the files assigned to this worker.
Server to client	FILE_HDR and DATA	Sends only this worker's files, from the validated cursor.
Client to server	ACK(cursor)	Commits a written chunk or file transition in this worker's session.
Server to client	FILE_END, TRANSFER_END	Completes a file or all work assigned to this worker.
Server to client	ERROR	Rejects an inconsistent assignment or checkpoint; BUSY can be retried.

Splitting the work. The server sorts the topic files and assigns file index i to worker $i \bmod K$. With $K = 4$, worker 0 receives indices 0, 4, 8 and so on. Splitting by whole files gives each file exactly one writer. Reassembly is simply writing each assigned file under its own name; there is no shared-file byte-range merge.

Recovery. Every worker keeps its own in-memory checkpoint and session ID. On a drop, only that worker reconnects and reports its cursor. The server verifies the topic, worker, count and committed/offered state using the Case 2 protocol. Other workers continue. The main client joins all workers and reports success only if every worker finishes.

Trade-offs. Each connection receives a full manifest and requires a socket, thread and session. File-level assignment can be uneven if file sizes vary. A topic with only one file cannot exploit several workers. If K exceeds the number of files, idle workers perform only their manifest and completion exchange. A dynamic file queue or byte-range splitting could improve load balance but is not implemented.

Both the server session table and worker checkpoints are volatile. Connection failures are recoverable; process restarts and power loss are outside the implemented guarantee. Output writes are not fsynced after every chunk.

3.4.2 Results

Table 9. Case 2 Enhanced on cane with four workers (three-run means)

$p\,(10^{-4})$	Injected failures	Control bytes	Time (s)	MiB/s
0.000	0.00	2,038,153	0.8781	66.10
2.313	1.33	2,343,150	0.8199	70.93
3.084	2.00	2,495,649	0.9060	63.96

Each run delivered all 4,863 images and every SHA-256 matched. With four workers, file counts were 1,216, 1,216, 1,216 and 1,215. Assigned bytes ranged from 14.40 to 14.66 MiB because image sizes differ. No payload was repeated under post-ACK injection. Full sweeps and overhead graphs are in `Results/Animals10/`.

Connection count was also measured at $p = 0.0000771$. Mean throughput was 26.60, 45.48, 71.12, 69.11 MiB/s for $K = 1, 2, 4, 8$. Corresponding mean completion times were 2.1797, 1.2985, 0.8177, 0.8383 s.

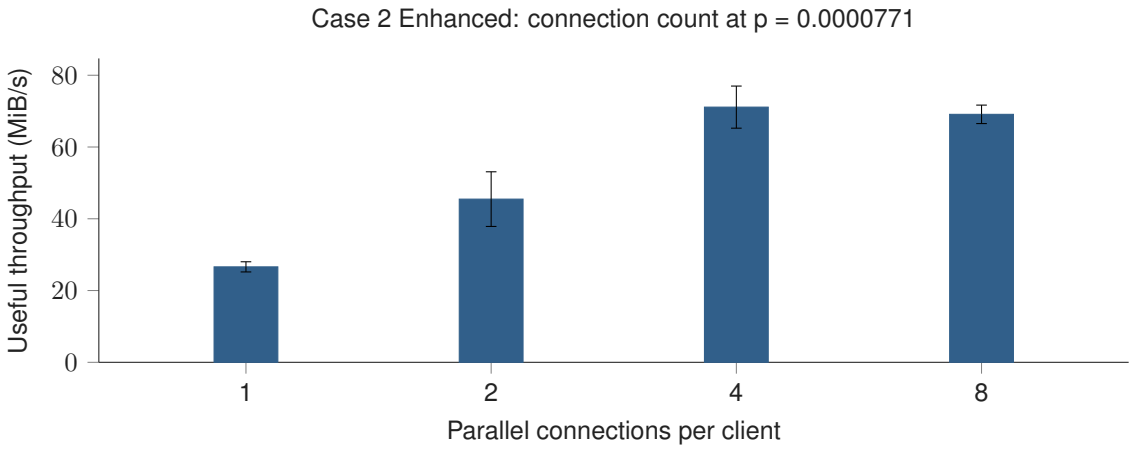


Figure 5. Measured enhanced throughput versus connection count; error bars show sample standard deviation.

3.4.3 Analysis

The ratio of eight-connection to one-connection mean throughput was 2.60 for this image topic. Workers can overlap socket waits, CRC checks and file operations, while CPU and storage remain shared. Each worker owns whole files, so uneven sizes and filesystem contention can limit scaling. The observed ratio is specific to this dataset and host.

Four-worker mean throughput was 63.96 MiB/s at $p = 0.0003084$ and 66.10 MiB/s without injection. Nonmonotonic differences, where present, can reflect sampling and scheduling; they are not evidence that faults improve performance. Clean four-worker control traffic was 2,038,153 bytes, compared with 1,352,050 for one worker, because each connection receives a complete manifest.

4. Summary of observations

Table 10. The implementations side by side

Implementation	After a failure	State kept	Main measured result
Part I, single-threaded	No recovery	No retained transfer checkpoint	27.91–30.13 MiB/s across 1, 2, 4 and 8 clients.
Part I, multithreaded	No recovery	No retained transfer checkpoint	29.75 and 129.58 MiB/s with 1 and 8 clients.
Case 1	Restart the topic	No recovery state	4.33 mean attempts; 118.99% extra payload at the highest p.
Case 2	Resume at cursor	Client cursor; server committed/offered session state	Zero repeated payload under injection; 2.0480 s at the highest p.
Case 2 Enhanced	Resume each worker	Independent client cursors and server sessions	Four workers: 63.96 MiB/s at the highest p; all images verified.

The experiments compare sequential and concurrent service, full restart and checkpoint recovery, and different connection counts on one complete Animals10 topic. Restart recovery repeats completed work; checkpoint recovery avoids repeating recorded chunks under the implemented fault policy. Parallel execution changes throughput and control costs. The raw observations and all graphs are supplied for independent inspection.

5. Deviations from the design and limitations

- The submitted design is implemented using C++17 containers with POSIX sockets and pthreads. Binary framing and a GET carrying the existing session ID implement the conceptual messages and RESUME operation. The protocol and report describe the actual code.
- All report measurements use the real Animals10 images on one Linux host. The required experiment with one server computer and two client computers remains unperformed. Local TCP processes do not satisfy that topology requirement; LAN scripts are included.
- Each configuration has three repetitions. The host is shared and page cache is not controlled. Reported means and sample deviations do not establish physical-network performance or a universal speed-up.
- Checkpoints are in memory. Process restart, power loss, changing source files, authenticated access and encrypted transport are not provided. CRC32 detects accidental chunk errors; it is not an authentication mechanism.
- All 28 integration checks passed, and a Case 2 ASan/UBSan recovery smoke run reported no error. LeakSanitizer was disabled because the sandbox restricts process inspection. These checks are evidence of tested behavior, not a proof that no bug exists.

Preparation note. The supplied Group 065/083 report is credited as the layout and organizational reference. Technical descriptions and results here correspond to the Group 097/112 implementation and logs. The package was prepared with AI assistance; the group should review the work and confirm actual contributions before submission.